

Learning Objectives

Learners should be able to...

- Filter stop words from text
- Find and tag parts of speech in text
- Stem and lemmatize text
- Write output to files

info

Make Sure to Know

Intermediate Python

Limitations

In this assignment, we only work with the NLTK library.

Frequency Distribution

Definition: Frequency Distribution

Now that we know how to create tokens from a text, one basic function performed on them is determining which token appears the most in a text. This is called frequency distribution, and the NLTK library has functions that can do this for us.

Use FreqDist

Import the applicable processes from the NLTK library used for forming frequency distributions in the exercises on this page.

```
from nltk.probability import FreqDist
from nltk.tokenize import word_tokenize
from nltk.corpus import inaugural
```

For this exercise, we will analyze the string contained in the variable called `sports_age`.

We need to tokenize the string by word before we can start forming frequency distributions.

```
sports_age = "Participating in team sports like softball,
              softball, basketball, tennis or volleyball doesn't have
              to stop based on your age."
sports_age_tokens = word_tokenize(sports_age)
```

Using `FreqDist`, we can find frequencies for each token in the text.

```
sports_age_freqdist = FreqDist(sports_age_tokens)
print(f"\nFrequency Results of Each Word Token in 'sports_age':
      \n{sports_age_freqdist}")
```

Using `most_common`, we can sort our output by the most common token frequencies.

Enter the below code, then select the TRY IT button to run the file.

```
sports_age_freqdist_most_common =  
    sports_age_freqdist.most_common()  
print("\nIn Order of Most Common Frequencies: ")  
for element in sports_age_freqdist_most_common:  
    print(element)
```

Your output should display as shown below.

```
Frequency Results of Each Word Token in 'sports_age':  
<FreqDist with 21 samples and 24 outcomes>
```

```
In Order of Most Common Frequencies:
```

```
(' ', 3)  
( 'softball', 2)  
( 'Participating', 1)  
( 'in', 1)  
( 'team', 1)  
( 'sports', 1)  
( 'like', 1)  
( 'basketball', 1)  
( 'tennis', 1)  
( 'or', 1)  
( 'volleyball', 1)  
( 'does', 1)  
( "n't", 1)  
( 'have', 1)  
( 'to', 1)  
( 'stop', 1)  
( 'based', 1)  
( 'on', 1)  
( 'your', 1)  
( 'age', 1)  
( '.', 1)
```

Using Frequency Distribution

We can combine frequency distribution with other aspects of tokens to help up narrow down our original text. Let's start by declaring the string variable called `hero` and calculating the frequency distribution for its tokens.

```
hero = "He saw that his hero and his commander were following  
quite a different train of thought."  
hero_freqdist = FreqDist(word_tokenize(hero))  
print(f"\nFrequency Results of Each Word Token in 'hero':  
{hero_freqdist}")
```

Your output should display as shown below.

```
Frequency Results of Each Word Token in 'hero': <FreqDist with  
16 samples and 17 outcomes>
```

We can use conditionals along with the frequency distribution of hero to retrieve specific token frequencies. In this case, we want all tokens whose length and frequency distribution are greater than 1.

```
conditional_hero = []  
for token in hero_freqdist.most_common():  
    if len(token[0]) > 1 and token[1] > 1:  
        conditional_hero.append(token)  
conditional_hero_no_dupes = set(conditional_hero)  
  
print(f"\nTokens longer than 1 character and with a frequency  
distribution greater than 1:  
{conditional_hero_no_dupes}")
```

Your output should display as shown below.

```
Frequency Results of Each Word Token in 'hero': <FreqDist with  
16 samples and 17 outcomes>
```

```
Tokens longer than 1 character and with a frequency distribution  
greater than 1: {('his', 2)}
```

challenge

Try this variation:

Using the corpus of text from all of the American presidential inaugural addresses, calculate the popularity of the word 'citizen' as a percentage.

Enter the code below, then select the TRY IT button to run the file.

```
freq_dist = FreqDist(inaugural.words())
percentage = 100 * (freq_dist.freq('citizen'))
print(f"\nPercentage of 'citizen' in the inaugural corpus:
      {percentage}%")
```

The expected output is shown below.

```
Percentage of 'citizen' in the inaugural corpus:
0.036625005722657145%
```

Filter Stop Words

Definition: Stop Words

One of the goals of NLP is extracting useful information from text for processing. Not every word is useful in this sense. These words are needed for sentences to sound a feel correct for humans, but they are not needed for NLP. These words are referred to as stop words.

- Stop words are words that we want to filter out, or ignore because they hold little meaning to the text. With stop words filtered out, especially when processing large bodies of text, we can more efficiently analyze the data.
- For example, words such as “a”, “the”, “are”, or “in” are frequently used in between words that hold greater meaning to the context of the text. Therefore, it is useful to filter out words like these, setting them as our stop words, so that we can more efficiently analyze the meaning of the text.

Filter Stop Words

Import the applicable processes from the NLTK library used for filtering stop words.

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
```

For this exercise, we will analyze the string contained in the variable called `choc_sentence`.

```
choc_sentence = 'If I had a chocolate bar, I would eat one piece  
of it every hour.'
```

We need to tokenize the string by word before we can start filtering by stop words; storing the sentence by word in the form of a list will allow us to further analyze the data, and filter by stop words.

```
choc_sentence_words = word_tokenize(choc_sentence)
```

We can create a set to filter out stop words written in English. Note that the command `stopwords.words('english')` only filters out stop words written in lower case English.

```
stop_words = set(stopwords.words('english'))
```

Using the casefold string method along with a for loop, we ignore the case of the letters while comparing each element in the list of words called `choc_sentence_words` to the set of stop words called `stop_words`. If the token is not in `stop_words` it is added to `choc_sentence_filtered`.

Enter the below code, then select the TRY IT button to run the file.

```
choc_sentence_filtered = [element for element in
                          choc_sentence_words if element.casefold() not in
                          stop_words]

print(f'\nChocolate Sentence, Original: {choc_sentence_words}')
print(f'\nChocolate Sentence, Filtered:
      {choc_sentence_filtered}')
```

Your output should display as shown below.

```
Chocolate Sentence, Original: ['If', 'I', 'had', 'a',
                              'chocolate', 'bar', ',', 'I', 'would', 'eat', 'one', 'piece',
                              'of', 'it', 'every', 'hour', '.']
Chocolate Sentence, Filtered: ['chocolate', 'bar', ',',
                              'would', 'eat', 'one', 'piece', 'every', 'hour', '.']
```

challenge

Try this variation:

We can also filter out stop words using the format shown in the code below to access the stopwords lexicon corpora. We just need to first import the `nltk` module in order to access corpora in one line of code.

Enter the below code, then select the TRY IT button to run the file.

```
import nltk
stop_words2 = nltk.corpus.stopwords.words('english')

choc_sentence_filtered2 = [element for element in
                           choc_sentence_words if element.casefold() not in
                           stop_words2]
print(f'\nChocolate Sentence, Filtered Using Import
      Variation: {choc_sentence_filtered2}')
```

Your output should display as shown below.

```
Chocolate Sentence, Filtered Using Import Variation:
['chocolate', 'bar', ',', 'would', 'eat', 'one', 'piece',
 'every', 'hour', '.']
```

More Examples

In this next exercise, we will work with the same string variable called `choc_sentence`, and we can reuse the variables called `stop_words` and `choc_sentence_words` from above.

We want to now filter out stop words, without including the words ‘i’ or ‘a’ from the filter. Utilizing sets, which cannot contain duplicate elements, we can remove each of these words from our list of stop words as shown in the code below.

```
stop_words_without_i_or_a = stop_words - {'i', 'a'}
```

From here, we can use the same process for filtering stop words from our text. Enter the below code, then select the TRY IT button to run the file.


```
choc_sentence_filtered_without_i_or_a = [element for element in
    choc_sentence_words if element.casefold() not in
    stop_words_without_i_or_a]
print(f"\nChocolate Sentence, Filtered Without Including 'i' or
    'a': {choc_sentence_filtered_without_i_or_a}")
```

Your output should display as shown below.

```
Chocolate Sentence, Filtered Without Including 'i' or 'a': ['I',
'a', 'chocolate', 'bar', ',', 'I', 'would', 'eat', 'one',
'piece', 'every', 'hour', '.']
```

challenge

Try this variation:

Write the code that produces the percentage of words in choc_sentence that are not stop words. Select the TRY IT button to run the file and check your work.

The expected output is shown below.

```
Percentage of Words That Are NOT Stop Words in
'choc_sentence': 66.15384615384615%
```

▼ Solution

One possible solution is shown below, where we found the percentage of words in choc_sentence that are not stop words by writing function not_stopwords_percentage that utilizes the len function.

```
def not_stopwords_percentage(data):
    content = [element for element in data if
        element.casefold() not in stop_words]
    percentage = 100 * (len(content) / len(data))
    return(percentage)

print(f"\nPercentage of Words That Are NOT Stop Words in
    'choc_sentence':
    {not_stopwords_percentage(choc_sentence)}%")
```

Tag Parts of Speech

Definition: Tagged Parts of Speech

Now that we have a list of important words, we can then tag them according to their part of speech.

- Tagging parts of speech (POS), or sometimes referred to as *grammatical tagging*, is the method computer applications apply to figure out the intended type of speech for a given word or phrase based on the context and situation used by the human.
- Nouns, adjectives, and foreign words are all examples of parts of speech.

▼ List of all POS tags

Enter the below command to see a list of all tags and their defined tagged parts of speech. Then, select the TRY IT button to view the results.

```
import nltk
print(nltk.help.upenn_tagset())
```

Listed below is a comprehensive list of all tags and their defined tagged parts of speech.

Tag	Part of Speech
CC	conjunction, coordinating
CD	numeral, cardinal
DT	determiner
EX	existential there
FW	foreign word
IN	preposition or conjunction, subordinating
JJ	adjective or numeral, ordinal
JJR	adjective, comparative
JJS	adjective, superlative
LS	list item marker
MD	modal auxiliary
NN	noun, common, singular or mass
NNP	noun, proper, singular
NNPS	noun, proper, plural
NNS	noun, common, plural
PDT	pre-determiner
POS	genitive marker
PRP	pronoun, personal

PRP\$	pronoun, possessive
RB	adverb
RBR	adverb, comparative
RBS	adverb, superlative
RP	particle
SYM	symbol
TO	“to” as preposition or infinitive marker
UH	interjection
VB	verb, base form
VBD	verb, past tense
VPG	verb, present participle or gerund
VBN	verb, past participle
VBP	verb, present tense, not 3rd person singular
VBZ	verb, present tense, 3rd person singular
WDT	WH-determiner
WP	WH-pronoun
WP\$	WH-pronoun, possessive
WRB	Wh-adverb

Tag Parts of Speech

Import the applicable functions from the NLTK library used for tagging in these exercises.

```
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk import pos_tag
from nltk.corpus import reuters
```

For this exercise, we will analyze the string contained in the variable called `outdoor_colors`.

```
outdoor_colors = 'The sky is blue, the moon is white, and the
                 leaves are green.'
```

We need to tokenize the string by word before we can start tagging parts of speech in words. Then, we can use the `pos_tag` function that we imported from the `nltk` module earlier to tag each word's part of speech in the list called `outdoor_colors_words`.

Enter the below code, then select the TRY IT button to run the file.

```
outdoor_colors_words = word_tokenize(outdoor_colors)
outdoor_colors_tagged_words = pos_tag(outdoor_colors_words)

print(f"List of Words in 'outdoor_colors':
      {outdoor_colors_words}")
print(f"List of Tagged Words in 'outdoor_colors':
      {outdoor_colors_tagged_words}")
```

Your output should display as shown below.

```
List of Words in 'outdoor_colors': ['The', 'sky', 'is', 'blue',
',', 'the', 'moon', 'is', 'white', ',', 'and', 'the', 'leaves',
'are', 'green', '.']
List of Tagged Words in 'outdoor_colors': [('The', 'DT'),
('sky', 'NN'), ('is', 'VBZ'), ('blue', 'JJ'), (',', ','),
('the', 'DT'), ('moon', 'NN'), ('is', 'VBZ'), ('white', 'JJ'),
(',', ','), ('and', 'CC'), ('the', 'DT'), ('leaves', 'NNS'),
('are', 'VBP'), ('green', 'JJ'), ('.', '.')]

```

▼ Did you notice?

```
[('The', 'DT'), ...]
```

The method called `pos_tag` places each word and its tagged part of speech into their own tuple within the main list.

Remember, tuples are immutable data structures.

challenge

Try this variation:

The string variable called `nonsense` contains all nonsense words. This string was taken from a verse written by John Lennon in *The Faulty Bagnose*. Calculate the parts of speech for these nonsensical words.

Enter the below code, then select the TRY IT button to run the file.

```
nonsense = 'The Mungle pilgriffs far away Religeorge too thee worled.  
          Sam fells on the waysock-side And somforbe on a gurled, With all  
          her faulty bagnose!'  
nonsense_words = word_tokenize(nonsense)  
nonsense_tagged_words = pos_tag(nonsense_words)  
  
print(f"\nList of Words in 'nonsense': {nonsense_words}")  
print(f"\nList of Tagged Words in 'nonsense' {nonsense_tagged_words}")
```

Your output should display as shown below.

```
List of Words in 'nonsense': ['The', 'Mungle', 'pilgriffs', 'far',  
'away', 'Religeorge', 'too', 'thee', 'worled', '.', 'Sam', 'fells',  
'on', 'the', 'waysock-side', 'And', 'somforbe', 'on', 'a', 'gurled',  
,',', 'With', 'all', 'her', 'faulty', 'bagnose', '!']  
List of Tagged Words in 'nonsense': [('The', 'DT'), ('Mungle', 'NNP'),  
( 'pilgriffs', 'NNS'), ('far', 'RB'), ('away', 'JJ'), ('Religeorge',  
'NNP'), ('too', 'RB'), ('thee', 'RB'), ('worled', 'VBN'), ('.', '.'),  
( 'Sam', 'NNP'), ('fells', 'VBZ'), ('on', 'IN'), ('the', 'DT'),  
( 'waysock-side', 'JJ'), ('And', 'CC'), ('somforbe', 'NN'), ('on', 'IN'),  
( 'a', 'DT'), ('gurled', 'VBN'), (',', ','), ('With', 'IN'), ('all',  
'DT'), ('her', 'PRP$'), ('faulty', 'NN'), ('bagnose', 'NN'), ('!', '!')]
```

As we can see, the computer is still able to identify the correct POS tags based on context even in strings that seem like nonsense to us.

Let's try it out!

In this next exercise, we will work with the string variable called `primary_colors`. We want to see what happens if we tag POS by sentence, rather than by word as we did above.

```
primary_colors = 'Blue is a primary color. And yellow is a primary color.  
Magenta is a primary color.'
```

From here, we can use the `sent_tokenize` method to create a list of sentences contained in `primary_colors`. Then, we will try using the `pos_tag` method to tag each sentence's POS in the list.

Predict what you think will happen, enter the below code, then select the TRY IT button to run the file.

```
primary_colors_sentences = sent_tokenize(primary_colors)  
primary_colors_tagged_sentences = pos_tag(primary_colors_sentences)  
  
print(f"\nList of Sentences in 'primary_colors':  
      {primary_colors_sentences}")  
print(f"\nList of Tagged Sentences in 'primary_colors':  
      {primary_colors_tagged_sentences}")
```

Your output should display as shown below.

```
List of Sentences in 'primary_colors': ['Blue is a primary color.', 'And  
yellow is a primary color.', 'Magenta is a primary color.']  
List of Tagged Sentences in 'primary_colors': [('Blue is a primary color.',  
'NNP'), ('And yellow is a primary color.', 'NNP'), ('Magenta is a primary  
color.', 'NNP')]
```



Output explained

```
List of Tagged Sentences in 'primary_colors': [('Blue is a primary color.',  
'NNP'), ('Yellow is a primary color.', 'NNP'), ('Magenta is a primary  
color.', 'NNP')]
```

Our output states that each sentence's tagged part of speech is a proper or singular noun.

Tag Part of Speech

NNP noun, proper, singular

This is because it treats each element in the list we use against the method `pos_tag` as one noun, even though each element is actually a sentence of words.

POS Tagging is meant to be applied to each word in a sentence, not the entire sentence because sentences do not individually represent a part of speech.

challenge

Try this variation:

We can also tag parts of speech in corpora with file identifiers, such as the Brown Corpus.

A file can be identified with a *file identifier*, which relatively represents the path to the file from the root directory.

Predict what you think will happen, enter the below code, then select the TRY IT button to run the file.

```
from nltk.corpus import brown, gutenberg

print(f'\nUsing tagged_words() on the Brown Corpus:
      {brown.tagged_words()}')
print('\nUsing tagged_words() on the Gutenberg Corpus: ')
print(gutenberg.tagged_words())
```

Your output should display as shown below.

```
Using tagged_words() on the Brown Corpus: [('The', 'AT'), ('Fulton',
'NP-TL'), ...]
Using tagged_words() on the Gutenberg Corpus:
Traceback (most recent call last):
print(gutenberg.tagged_words())
AttributeError: 'PlaintextCorpusReader' object has no attribute
'tagged_words'
```



What does the error mean?

```
Using tagged_words() on the Gutenberg Corpus:
Traceback (most recent call last):
print(gutenberg.tagged_words())
AttributeError: 'PlaintextCorpusReader' object has no attribute
'tagged_words'
```

- Several corpora use *file identifiers* to index their content, as the Brown Corpus does.
- The Gutenberg Corpus does not use file identifiers to index its content.

Important: Before moving on, make sure to comment out or delete the following lines from your code.

```
print(gutenberg.tagged_words())
```


Stem Text

Definition: Stemming

Another way we can prepare a text for processing is through stemming.

- Stemming is used to reduce plain text words to their *stem*, or *root* word (also known as the *base* word). Using the process of stemming allows us to better understand the meaning of each word.
- For example, the words “egocentric”, “egotist” and “egomaniac” all possess the same stem word: “ego”. Stemming removes the suffix or prefix on a word to more easily isolate its meaning.

Two commonly-used stemmers are the `PorterStemmer` and the `SnowballStemmer`.

- *The PorterStemmer* is designed to remove inflexional endings from words and is the most commonly-used stemmer. It takes a more soft approach to stemming than the `SnowballStemmer`.

- *The SnowballStemmer* (often referred to as the *Porter2Stemmer*) is also a commonly used stemmer, as it was designed to improve the original `PorterStemmer`. This stemmer has a faster processing time and it takes a more aggressive approach to stemming compared to the `PorterStemmer`

▼ Over- and understemming

It is important to note that stemming is not always the best approach to understanding the contextual meaning of words, which is why we also use a process called lemmatization (which we will discuss on the next page of this assignment).

- *Understemming* occurs when two or more words are supposed to have the same stem word, but the computer tells us that they have different stem words.
 - For example, the words “alumnus” and “alumni” should have the same stem word. However, the stem word for “alumnus” is `alumnus` (using the `PorterStemmer`) or `alumnus` (using the `SnowballStemmer`) depending on the stemmer used, while the stem word for “alumni” is `alumni`. Understemming is also referred to as being a false negative.
- *Overstemming* occurs when a word is reduced too far, so that its stem is no longer correct or helpful for analysis.
 - For example, the words “university” and “universe” do not relate to the same meaning, yet they have the same stem word: `univers`. Overstemming is also referred to as being a false positive.

PorterStemmer

Import the applicable processes from the NLTK library used for tagging parts of speech in these exercises.

```
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk import LancasterStemmer
from nltk.stem import SnowballStemmer
```

For this exercise, we will analyze the string contained in the variable called `paper_grade`.

```
paper_grade = 'I am fairly certain that I will get an A on my paper.'
```

We need to tokenize the string by word before we can start finding the stem of each word. Then, we can use the `PorterStemmer` method that we imported from the `nltk.stem` module earlier to create the stemmer called `porter_stemmer`.

```
paper_grade_words = word_tokenize(paper_grade)
porter_stemmer = PorterStemmer()
```

In the code below, we use list comprehension and the `stem` method to create a list of each word's stem. Enter the below code, then select the TRY IT button to run the file.

```
paper_grade_stemmed_words = [porter_stemmer.stem(element) for
                             element in paper_grade_words]
print(f"\nList of Words in 'paper_grade': {paper_grade_words}")
print(f"\nList of Stems in 'paper_grade' using the
      PorterStemmer() Method: {paper_grade_stemmed_words}")
```

Your output should display as shown below.

```
List of Words in 'paper_grade': ['I', 'am', 'fairly', 'certain',
'that', 'I', 'will', 'get', 'an', 'A', 'on', 'my', 'paper', '.']

List of Stems in 'paper_grade' using the PorterStemmer() Method:
['i', 'am', 'fairli', 'certain', 'that', 'i', 'will', 'get',
'an', 'a', 'on', 'my', 'paper', '.']
```

challenge

Try this variation:

The most aggressive stemmer used in the NLTK library is by far the LancasterStemmer. We can use the LancasterStemmer method that we imported from the nltk module earlier to create the stemmer called `lancaster_stemmer`.

Enter the below code, then select the TRY IT button to run the file.

```
lancaster_stemmer = LancasterStemmer()
paper_grade_stemmed_words2 =
    [lancaster_stemmer.stem(element) for element in
     paper_grade_words]
print(f"\nList of Stems in 'paper_grade' using the
      LancasterStemmer() Method:
      {paper_grade_stemmed_words2}")
```

Your output should display as shown below.

```
List of Stems in 'paper_grade' using the LancasterStemmer()
Method: ['i', 'am', 'fair', 'certain', 'that', 'i', 'wil',
        'get', 'an', 'a', 'on', 'my', 'pap', '.']
```

As we can see, even the word ‘paper’ was minimized down to the stem: `pap`. Very aggressive.

SnowballStemmer

In this next exercise, we will continue to work with the variable called `paper_grade` so that we can compare stemmer results.

We can use the SnowballStemmer method that we imported from the `nltk.stem` module earlier to create the stemmer called `snowball_stemmer`. Then, we will use this stemmer to compare our string variable to stems stored in the English language.

```
snowball_stemmer = SnowballStemmer('english')
```

In the code below, we use list comprehension to create a list of each word’s stem. Enter the below code, then select the TRY IT button to run the file.

```
paper_grade_stemmed_words3 = [snowball_stemmer.stem(element) for
                              element in paper_grade_words]
print(f"\nList of Stems in 'paper_grade' using the
      SnowballStemmer() Method: {paper_grade_stemmed_words3}")
```

Your output should display as shown below.

```
List of Stems in 'paper_grade' using the SnowballStemmer()
Method: ['i', 'am', 'fair', 'certain', 'that', 'i', 'will',
        'get', 'an', 'a', 'on', 'my', 'paper', '.']
```

▼ **Did you notice?**

```
...using the PorterStemmer() Method: ['i', 'am', 'fairli', ...]
```

```
...using the SnowballStemmer() Method: ['i', 'am', 'fair', ...]
```

The word “fairly” from our example string displays a great example of the difference in how these two stemmers work. The SnowballStemmer is the more aggressive stemmer out of the two.

challenge

Try this variation:

We can also use the SnowballStemmer to ignore stemming stop words.
Syntax:

```
stemmer_name = SnowballStemmer('english', ignore_stopwords =
True/False)
```

Enter the below code, then select the TRY IT button to run the file.

```
bread = 'Before we had rubber erasers, we erased graphite
with rolled-up white bread.'

bread_stemmed_words = [snowball_stemmer.stem(element) for
element in word_tokenize(bread)]
print(f"\nList of Stems in 'bread' using the
SnowballStemmer() Method: {bread_stemmed_words}")

snowball_stemmer_ignore_stopwords =
SnowballStemmer('english', ignore_stopwords = True)
bread_stemmed_words_ignore_stopwords =
[snowball_stemmer_ignore_stopwords.stem(element) for
element in word_tokenize(bread)]
print(f"\nList of Stems in 'bread' using the
SnowballStemmer() Method Without Stemming Stop
Words: {bread_stemmed_words_ignore_stopwords}")
```

Your output should display as shown below.

```
List of Stems in 'bread' using the SnowballStemmer() Method:
['befor', 'we', 'had', 'rubber', 'eras', '', 'we', 'eras',
'graphit', 'with', 'rolled-up', 'white', 'bread', '.']

List of Stems in 'bread' using the SnowballStemmer() Method
Without Stemming Stop Words: ['before', 'we', 'had',
'rubber', 'eras', '', 'we', 'eras', 'graphit', 'with',
'rolled-up', 'white', 'bread', '.']
```

Lemmatize Text

Definition: Lemmatization

Like stemming, lemmatization reduces words to their most basic meaning. The difference is that stemming will give us the *root* (or stem) of the original word even if it is broken (e.g., the root of the word “every” comes out as “everi” when stemming), whereas lemmatizing will give us a word that represents a grouping of words (e.g., the lemma of the word “every” is still “every”).

- The word lemmatization comes from the words *lemma* and *lexeme*.
- A lemma is a word that represents a grouping of words.
- A lexeme is a grouping of words.
- When we use lemmatization on words, we are reducing them to their lemma.
- For example, the lemma of the word “chocolates” is “chocolate”.

WordNetLemmatizer

The NLTK library contains a lexicon corpus called wordnet which can lemmatize text. We already downloaded this corpus in the first assignment of the module.

Import the applicable processes from the NLTK library used for lemmatizing in these exercises. We need to import the WordNetLemmatizer from the `nltk.stem` module.

```
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from pprint import pprint
from nltk import pos_tag
from nltk.corpus import wordnet
```

For this exercise, we will analyze the string contained in the variable called `positive_quote`.

```
positive_quote = 'Better days are on their way, so stay  
positive.'
```

We need to tokenize the string by word before we can start finding the lemma of each word. Then, we can use the `WordNetLemmatizer` method that we imported from the `nltk.stem` module earlier to create the lemmatizer

called lemmatizer.

```
positive_quote_words = word_tokenize(positive_quote)
lemmatizer = WordNetLemmatizer()
```

In the code below, we use a list comprehension and the lemmatize method to create a list of each word's lemma. Enter the below code, then select the TRY IT button to run the file.

```
positive_quote_lemmatized_words = [lemmatizer.lemmatize(element)
    for element in positive_quote_words]
print(f"\nList of Words in 'positive_quote':
    {positive_quote_words}")
print(f"List of Lemmas in 'positive_quote':
    {positive_quote_lemmatized_words}")
```

Your output should display as shown below.

```
List of Words in 'positive_quote': ['Better', 'days', 'are',
    'on', 'their', 'way', ',', 'so', 'stay', 'positive', '.']

List of Lemmas in 'positive_quote': ['Better', 'day', 'are',
    'on', 'their', 'way', ',', 'so', 'stay', 'positive', '.']
```

▼ Did you notice?

The computer told us that the lemma of the word “better” is “better”.

- This is because the WordNetLemmatizer method automatically assumes that all words are nouns when analyzing input.
- If we tell the computer that the word “better” is actually written as an adjective, then we will receive different output. We will practice this next.

challenge

Try this variation:

Remember that the `WordNetLemmatizer` method automatically assumes that all words are nouns when analyzing input. To tell the computer what part of speech the word “better” was written as, we can use the below command, where we will lemmatize only the word “better” and tell the computer that it is an adjective (“a” stands for adjective).

Important: We can find the lemma of a word using the correct part of speech by entering the following syntax.

```
lemmatizer_name.lemmatize('string', pos = 'pos_tag')
```

Enter the below code, then select the TRY IT button to run the file.

```
print(f"\nLemma of the Noun 'better' (default: pos = 'n'):\n{lemmatizer.lemmatize('better')}")\nprint(f"Lemma of the Adjective 'better':\n{lemmatizer.lemmatize('better', pos='a')}")
```

Your output should display as shown below.

```
Lemma of the Noun 'better' (default: pos = 'n'): better\n\nLemma of the Adjective 'better': good
```

Now, the computer tells us that the lemma of the word “better” written as an adjective is “good”. This is because the word “better” is a form of the word “good”.

Lemmatization Functions

In this next exercise, we will continue to work with the variable called `positive_quote` so that we can compare our results to those exercises above.

First, we need to create a function that changes each word’s POS tag into an acceptable format for wordnet. Each word’s POS tag is contained in the variable called `tag` using upper-case. We can use a dictionary to translate the POS tag name to an understandable format for wordnet, as shown in the code below.

```
def wordnet_pos(word):
    tag = pos_tag([word])[0][1][0].upper()
    tag_dict = {"N": wordnet.NOUN,
                "J": wordnet.ADJ,
                "V": wordnet.VERB,
                "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)
```

▼ Understanding the function

Remember, the `pos_tag` function takes and returns a list. So word has to become `[word]` for tagging purposes. The return value is a list that contains a tuple with two items. The `[0][1][0]` means we are looking at the first character of the POS tag string (the second `[0]`) of the second item in a tuple (the `[1]`) which is the first element of a list (the first `[0]`). If this first character is not found in the dictionary, return `wordnet.NOUN` instead.

Enter the below code, then select the TRY IT button to run the file.

```
print(f"\nList of Words in 'positive_quote':
      {positive_quote_words}")
print(f"List of Lemmas (Using POS Tags) in 'positive_quote':
      {[lemmatizer.lemmatize(word, wordnet_pos(word)) for word
       in word_tokenize(positive_quote)]}")
```

Your output should display as shown below.

```
List of Words in 'positive_quote': ['Better', 'days', 'are',
                                     'on', 'their', 'way', ',', 'so', 'stay', 'positive', '.']
```

```
List of Lemmas (Using POS Tags) in 'positive_quote': ['Better',
                                                         'day', 'be', 'on', 'their', 'way', ',', 'so', 'stay',
                                                         'positive', '.']
```

challenge

Try this variation:

Use the 'pretty print' module, `pprint`, to aid your output's readability. Remember that we imported the `pprint` module at the beginning of the page.

Enter the below code, then select the TRY IT button to run the file.

```
def pos_lemmatize_pprint(text):
    words = word_tokenize(text)
    lemmatized_words = [lemmatizer.lemmatize(word,
        wordnet_pos(word)) for word in words]

    print('Original Word : Lemma')
    pprint({words[word] : lemmatized_words[word] for word in
        range(len(lemmatized_words))}, indent = 10, depth =
        5)

print("\nList of Lemmas (Using POS Tags) in 'positive_quote'
    with pprint:")
pos_lemmatize_pprint(positive_quote)
```

Your output should display as shown below.

```
List of Lemmas (Using POS Tags) in 'positive_quote' with
pprint:
Original Word : Lemma
{
    ',': ',',
    '.': '.',
    'Better': 'Better',
    'are': 'be',
    'days': 'day',
    'on': 'on',
    'positive': 'positive',
    'so': 'so',
    'stay': 'stay',
    'their': 'their',
    'way': 'way'}
```

▼ Did you notice?

Since the pprint method uses a dictionary to structure its output, the order of each word in the sentence is no longer preserved.

Formatting Text & Writing to Files

Formatting Text

Performing NLP on large bodies of text means that the output can be rather long. We can format how the output is shown to increase legibility. We can add padding to f-strings so the text is lined up in columns. Let's start by looking at numerical values. Padding is done with a `:`. The number after the colon represents the padding (empty spaces) to the left of the value.

Enter the below code, then select the TRY IT button to run the file.

```
num = 7

print(f'{num:03}')
print(f'{num:3}')
```

Notice the difference the extra `0` makes with the padding. Without it, spaces are used. With it, the spaces replace the zeros. Watch what happens when we try the same thing with a string. Note, the padding needs to be longer than the string. We should see a padding of 2 since 5 (the padding) minus 3 (the length of `dog`) is 2.

```
str = 'dog'

print(f'{str:5}')
```

Nothing seems to have changed. Padding for strings is to the right by default. In addition, using `:05` will result in an error for strings. That syntax only works for numbers. Instead, we can use the `:>5` syntax to have a padding of 5 be to the left of the string.

```
str = 'dog'

print(f'{str}')
print(f'{str:>5}')
```

Notice the space between the `:` and the `>`. Whatever goes between these two characters will be used for the padding. In addition, using `<` or `>` determines where the padding goes in relation to the string.

challenge

Try this variation:

Add non-space padding to the left and right of the `str` variable.

```
str = 'dog'

print(f'{str:~<5}')
print(f'{str:~>5}')
```

Let's put these formatting skills to use. We are going to generate a table of inaugural addresses by US presidents. Each row should contain the year of the speech, who gave it, and the length of speech by word count. Each column should be padded so that everything lines up nicely. We are going to focus on the formatting, so we are going to import the `generate_table()` function that calls several helper functions that parse the data. The dropdown below has the code for the helper functions if you are interested.

▼ Helper Functions

Functions used to parse the inaugural address data.

```

from nltk.corpus import inaugural

def parse_year(addresses):
    return [year[0:4] for year in addresses]

def parse_president(addresses):
    presidents = []
    for president in addresses:
        dot_index = president.find('.')
        name = president[5:dot_index]
        presidents.append(name)
    return presidents

def calculate_length(addresses):
    lengths = []
    for address in addresses:
        words_punctuation = inaugural.words(address)
        just_words = [word for word in words_punctuation if
word.isalpha()]
        length = len(just_words)
        lengths.append(length)
    return lengths

def combine_data(years, presidents, lengths):
    return tuple(zip(years, presidents, lengths))

def generate_table(addresses):
    years = parse_year(addresses)
    presidents = parse_president(addresses)
    lengths = calculate_length(addresses)
    table = combine_data(years, presidents, lengths)
    return table

```

Import the `inaugural` corpus and the `generate_table()` function from the helper module. The `fileids()` method returns a list of strings like `1789-Washington.txt`. The `generate_table()` function will parse this list and read the speech from each file and return a nested tuple that contains the year, the president, and the length (in words) of their inaugural address.

The `print_table()` function will print the contents of the nested tuple in a formatted manner. Start by giving a title to each column. Add a padding of 8 to the year, which means the year will be followed by 4 spaces. The column for president names has the most variety in terms of length. Using a padding of 14 will produce 4 spaces to the right of the longest name. The

column for length does not need any padding as it is the last column. Iterate over the nested tuple and print each item with the same padding as the titles.

Enter the below code, then select the TRY IT button to run the file.

```
from nltk.corpus import inaugural
from helper import generate_table

def print_table(data):
    print(f"{'Year': <8}{'President': <14}{'Length'}")
    print(f"{'----': <8}{'-----': <14}{'-----'}")
    for d in data:
        print(f'{{d[0]: <8}}{{d[1]: <14}}{{d[2]}}')

inaugural_addresses = inaugural.fileids()
info = generate_table(inaugural_addresses)
print_table(info)
```

Here is the output at the time of writing:

Year	President	Length
----	-----	-----
1789	Washington	1430
1793	Washington	135
1797	Adams	2322
1801	Jefferson	1732
1805	Jefferson	2168
1809	Madison	1177
1813	Madison	1211
1817	Monroe	3378
1821	Monroe	4464
1825	Adams	2917
1829	Jackson	1128
1833	Jackson	1177
1837	VanBuren	3846
1841	Harrison	8469
1845	Polk	4813
1849	Taylor	1091
1853	Pierce	3343
1857	Buchanan	2836
1861	Lincoln	3635
1865	Lincoln	705
1869	Grant	1132
1873	Grant	1339
1877	Hayes	2489
1881	Garfield	2986

1885	Cleveland	1690
1889	Harrison	4397
1893	Cleveland	2028
1897	McKinley	3970
1901	McKinley	2209
1905	Roosevelt	990
1909	Taft	5437
1913	Wilson	1711
1917	Wilson	1531
1921	Harding	3346
1925	Coolidge	4056
1929	Hoover	3573
1933	Roosevelt	1883
1937	Roosevelt	1823
1941	Roosevelt	1346
1945	Roosevelt	558
1949	Truman	2283
1953	Eisenhower	2461
1957	Eisenhower	1664
1961	Kennedy	1367
1965	Johnson	1491
1969	Nixon	2130
1973	Nixon	1816
1977	Carter	1227
1981	Reagan	2444
1985	Reagan	2572
1989	Bush	2335
1993	Clinton	1609
1997	Clinton	2162
2001	Bush	1593
2005	Bush	2087
2009	Obama	2412
2013	Obama	2120
2017	Trump	1466
2021	Biden	2589

Writing to Files

We can create a file called `output.txt` to record our output instead of printing to the screen. There are several ways to write to a file in Python, but we are going to use the `with open` syntax. The main benefit of this is that Python will automatically close the file for you. Otherwise you have to manually close the file, something that can cause problems if you forget to do.

Here is the general syntax for writing to a file. Pass the file name as a string as well as the mode to the open function. Python defaults to read mode for files. Since we want to write to a file, we have to include 'w'. The `as writer` means we can refer to the file as `writer` as we process the data and write to the file. Use the `write` method to write a string to the file, or use the `writelines` to write a sequence to the file. If the output file does not exist, Python will create it for you. When you exit the `with` block, Python closes the file.

```
with open('file.txt', 'w') as writer:
    # Process the data
    # and write to
    # the file here
```

We are going to write each word from President Bush's 1989 Inaugural Address to the `bush_1989_address.txt` file. The file will reside in the `exercises` directory. We do not want any punctuation, nor do we want any duplicate words. Each word should appear on its own line, and the words should be in alphabetical order. Start by importing the `inaugural` corpus. Then open a text file in write mode. Get all of the words from the address. Using a list comprehension and ternary operator, create a new list of lowercase words if the word is alphanumeric (i.e. not punctuation). Remove duplicate words by creating a set from the list of lowercase words. Using another list comprehension, create a list of each word from the sorted set plus a newline character. Finally, write these lines to the file.

Enter the below code, then select the TRY IT button to run the file.

```
from nltk.corpus import inaugural

with open('exercises/bush_1989_address.txt', 'w') as writer:
    address_words = inaugural.words('1989-Bush.txt')
    lowercase_list = [word.lower() for word in address_words if
                      word.isalpha()]
    lowercase_set = set(lowercase_list)
    writer.writelines([word + '\n' for word in
                      sorted(lowercase_set)])
```

Use the link below to open the newly created file. You should see a list of every word from the address. The text file should have 743 words. The first word is `a`, and the last word is `your`.

challenge

Try this variation:

Instead of writing each word to a file, create a script that writes each sentence to the `bush_sentences.txt` file. Remember, the text file should reside in the `exercises` directory.

▼ Solution

Here is one possible solution:

```
from nltk.corpus import inaugural

with open('exercises/bush_sentences.txt', 'w') as writer:
    address_sentences_list = inaugural.sents('1989-Bush.txt')
    address_sentences = [f"{' '.join(sentence)}\n" for
                        sentence in address_sentences_list]
    writer.writelines(address_sentences)
```

- Open the file `exercises/bush_sentences.txt` in write mode.
- Use the `sents` method to get the sentences from the corpus. This is a 2D list where the inner lists contain each word for each sentence.
- Use the `join` method with a space to turn the inner list of words into a string representing the complete sentence. This solution is not perfect in that there are spaces to the left and right of punctuation. We also need to add a newline character so each sentence appears on its own line.
- Write these lines to the file.

Formative Assessment 1

Formative Assessment 2
